

ThreadSafeQueueLib

High-Performance Concurrent Queue Library in Modern C++

This document serves as a consolidated reference guide for all the major concurrency concepts, architectural decisions, benchmarking observations, and low-level C++ optimizations applied throughout the ThreadSafeQueueLib project.

Contents

1	Concurrency Models: Lock-Free vs. Blocking	2
2	Advanced C++ Atomics & Memory Ordering	3
3	Combating False Sharing (<code>alignas(64)</code>)	4
4	Power-of-Two Capacity & Fast Modulo (Bitmasking)	5
5	Avoiding Dynamic Allocation (Ring Buffers)	6
6	The ABA Problem & Sequence Numbers	6
7	The Stub/Sentinel Node Pattern	7
8	Nothrow Guarantees via Static Asserts	8
9	In-Place Construction (<code>try_emplace</code>)	8
10	Benchmark Results Analysis	10
10.1	Single-Producer Single-Consumer (SPSC)	10
10.2	Multi-Producer Single-Consumer (MPSC)	12
10.3	Lock-Free MPMC Bounded	12
10.4	Blocking MPMC	13
11	Final Takeaways	14

1 Concurrency Models: Lock-Free vs. Blocking

Modern concurrent queue designs typically fall into two broad categories:

- Blocking synchronization
- Lock-free synchronization

Both models optimize for fundamentally different goals and behave very differently under contention.

Blocking (Mutexes & Condition Variables)

Blocking queues use operating-system-backed synchronization primitives such as:

- `std::mutex`
- `std::condition_variable`

When the queue becomes empty or full, threads voluntarily sleep and yield execution back to the operating system scheduler.

Advantages

- **Excellent CPU Efficiency**
Sleeping threads consume essentially zero CPU resources while idle.
- **Good Power Efficiency**
Since threads are descheduled rather than spinning continuously, overall power consumption remains low.
- **Fairness**
Operating system schedulers generally provide fair access to resources across competing threads.
- **Ideal for Bursty Workloads**
Blocking queues work extremely well for systems where work arrives intermittently rather than continuously.

Disadvantages

Blocking synchronization introduces substantial operating system overhead.

- Threads may enter the kernel scheduler frequently
- Context switches are expensive
- Waking sleeping threads incurs latency
- Cache locality may be destroyed after descheduling

A single context switch can cost thousands of CPU cycles.

Lock-Free (Spinning)

Lock-free queues avoid operating system scheduling entirely.

Instead of sleeping, threads repeatedly retry operations using atomic primitives:

```
while (!try_push(value)) {  
    // spin  
}
```

Advantages

- **Extremely Low Latency**
No thread suspension or kernel scheduling is required.
- **High Throughput**
Operations remain entirely in user space and avoid expensive mutex handoffs.
- **Excellent for Hot Paths**
Lock-free algorithms excel in continuously active systems such as:
 - Trading systems
 - Telemetry pipelines
 - Real-time processing engines

Disadvantages

- Threads continuously burn CPU cycles while spinning
- Poor power efficiency
- Heavy cache coherency pressure under contention
- Atomic retry loops may scale poorly

Lock-free algorithms eliminate locks, but they do not eliminate contention.

2 Advanced C++ Atomics & Memory Ordering

Modern CPUs aggressively reorder instructions internally for performance.

Without explicit synchronization, different threads may observe memory operations in completely different orders.

C++ atomics provide tools to control these visibility guarantees.

Sequential Consistency

The strongest memory model is:

```
std::memory_order_seq_cst
```

This guarantees a globally consistent ordering of atomic operations across all threads.

Problem

Sequential consistency introduces strong memory fences which may:

- Stall CPU pipelines
- Restrict instruction reordering
- Increase synchronization traffic
- Reduce out-of-order execution efficiency

High-performance concurrent structures therefore avoid it whenever possible.

`memory_order_relaxed`

Used when only atomicity is required without synchronization.

Typical use cases:

- Statistics counters
- Retry loops
- Non-synchronizing metadata

Relaxed ordering allows maximum compiler and CPU optimization freedom.

`memory_order_acquire` / `memory_order_release`

Acquire-release semantics establish synchronization boundaries.

Release Store

A release operation guarantees that all prior writes become visible before the atomic store itself becomes visible.

Acquire Load

An acquire load guarantees that all writes published before the matching release become visible afterward.

Together they create a synchronization relationship:

Producer Release $\rightarrow$ Consumer Acquire

This is heavily used in lock-free queue publication protocols.

3 Combating False Sharing (`alignas(64)`)

The Problem

Modern CPUs operate on cache lines, typically 64 bytes wide.

If two independent variables reside on the same cache line and different threads modify them simultaneously, the CPU repeatedly invalidates cache ownership across cores.

This phenomenon is called:

False Sharing

Even though the variables themselves are logically unrelated, the hardware treats the entire cache line as shared mutable state.

The Fix

Critical atomics are padded and aligned:

```
alignas(64)
```

This forces frequently modified variables onto separate cache lines.

Benefits

- Reduced cache invalidation traffic
- Improved scalability
- Lower inter-core synchronization overhead
- Better throughput under contention

False sharing alone can reduce concurrent throughput by orders of magnitude.

4 Power-of-Two Capacity & Fast Modulo (Bitmasking)

The Problem

Ring buffers require wraparound indexing:

$$index = pos \% Capacity$$

Integer modulo operations internally perform division, which is relatively expensive on modern CPUs.

Power-of-Two Optimization

If:

$$(Capacity \& (Capacity - 1)) = 0$$

then capacity is a power of two.

Modulo can then be replaced by:

$$index = pos \& (Capacity - 1)$$

This converts division into a single bitwise instruction.

Benefits

- Lower instruction latency
- Faster hot-path execution
- Better branch prediction behavior
- Reduced arithmetic overhead

This optimization is extremely common in high-performance circular buffer implementations.

5 Avoiding Dynamic Allocation (Ring Buffers)

Dynamic memory allocation is one of the largest hidden costs in concurrent systems.

Heap allocators internally maintain synchronization structures and metadata.

Under heavy contention, allocator overhead becomes substantial.

Problems with Dynamic Allocation

- Allocator synchronization
- Heap fragmentation
- Unpredictable latency
- Kernel interaction
- Poor cache locality
- Increased TLB pressure

Bounded Ring Buffers

Bounded queues allocate all storage upfront.

Elements are stored inside contiguous arrays rather than scattered heap nodes.

Benefits

- Excellent cache locality
- Predictable latency
- Hardware prefetching efficiency
- Deterministic memory usage
- No allocator bottlenecks

This is a major reason why bounded queues vastly outperform linked-list structures.

6 The ABA Problem & Sequence Numbers

The ABA Problem

Suppose a thread observes value:

$$A$$

Then another thread changes it:

$$A \rightarrow B \rightarrow A$$

The original thread later resumes and incorrectly assumes nothing changed because the value appears identical.

This is the classic ABA problem.

Why It Is Dangerous

CAS-based algorithms only compare values.

They cannot determine whether intermediate transitions occurred.

This may cause:

- Use-after-free bugs
- Corrupted queue state
- Lost updates

The Fix: Sequence Numbers

Each slot maintains a continuously increasing sequence number representing its generation.

Threads therefore compare:

- Slot state
- Generation/version

rather than raw pointer identity alone.

This technique is heavily used inside:

- `lockfree_mpmc_bounded`

7 The Stub/Sentinel Node Pattern

Linked-list based queues use a permanent dummy node called a sentinel node.

Used in:

- `lockfree_spsc_unbounded`
- `lockfree_mpsc_unbounded`

Why Use a Sentinel Node?

Without a dummy node, queues constantly transition between:

- Empty
- Non-empty

This creates difficult edge cases and synchronization hazards.

Benefits

With a permanent stub node:

- `head` always points somewhere valid
- `tail` always points somewhere valid
- Empty-state handling becomes simpler
- `nullptr` races disappear
- Lock-free transitions become cleaner

This greatly simplifies algorithm correctness.

8 Nothrow Guarantees via Static Asserts

Lock-free algorithms are extremely sensitive to exceptions.

If an exception occurs mid-operation, internal queue state may become permanently corrupted.

Compile-Time Enforcement

The library enforces:

```
static_assert(std::is_nothrow_move_assignable_v<T>);
```

This guarantees that object movement inside critical lock-free regions cannot throw exceptions.

Why It Matters

Without this guarantee:

- Partially published objects may exist
- Sequence numbers may become inconsistent
- Slots may never recover correctly

Compile-time enforcement avoids these failure modes entirely.

9 In-Place Construction (`try_emplace`)

The Problem

Traditional push operations often require:

1. Constructing an object externally
2. Moving/copying it into the queue

This introduces unnecessary temporary objects and memory movement.

The Fix

```
try_emplace(Args&&... args)
```

uses:

- Perfect forwarding
- `std::forward`
- Placement `new`

to construct the object directly inside queue storage.

Benefits

- Eliminates temporary allocations
- Avoids unnecessary move/copy operations
- Improves cache efficiency
- Reduces instruction count

In-place construction is particularly important for large or move-expensive types.

10 Benchmark Results Analysis

This section provides a detailed breakdown of the performance characteristics observed from the Google Benchmark suite executed on a 16-core system.

Reference Benchmark Results

Benchmark	Time(ns)	CPU(ns)	Iterations	Items/sec	Ops/sec
BM_SPSC_UNBOUNDED	1552548400	0.000	1	6.44M/s	6.44M/s
BM_SPSC_BOUNDED	71895508	0.000	12	139.09M/s	139.09M/s
BM_MPSC/1	348759500	0.000	2	5.73M/s	5.73M/s
BM_MPSC/2	603713400	0.000	1	6.62M/s	6.62M/s
BM_MPSC/4	1153064000	0.000	1	6.93M/s	6.93M/s
BM_MPSC/8	2324944000	0.000	1	6.88M/s	6.88M/s
BM_LOCKFREE_MPMC_BOUNDED/2	31370759	0.000	22	31.87M/s	31.87M/s
BM_LOCKFREE_MPMC_BOUNDED/4	203394600	0.000	4	9.83M/s	9.83M/s
BM_LOCKFREE_MPMC_BOUNDED/8	472495700	0.000	2	8.46M/s	8.46M/s
BM_LOCKFREE_MPMC_BOUNDED/16	1389355600	0.000	1	5.75M/s	5.75M/s
BM_BLOCKING_MPMC/2	316236500	0.000	2	3.16M/s	3.16M/s
BM_BLOCKING_MPMC/4	816578600	0.000	1	2.44M/s	2.44M/s
BM_BLOCKING_MPMC/8	2578731000	0.000	1	1.55M/s	1.55M/s
BM_BLOCKING_MPMC/16	7226145200	0.000	1	1.10M/s	1.10M/s

10.1 Single-Producer Single-Consumer (SPSC)

BM_SPSC_BOUNDED — **139.09 Million Ops/sec**

This queue achieves extremely high throughput because it combines several ideal properties for modern CPU architectures.

- **Contiguous Memory Layout**

The bounded SPSC queue stores all elements inside a fixed-size contiguous ring buffer. Since adjacent elements reside near each other in memory, CPU cache lines are utilized extremely efficiently. Sequential accesses allow the processor’s hardware prefetcher to accurately predict future memory accesses and load upcoming cache lines proactively into L1/L2 cache before they are needed.

- **Zero Dynamic Allocation**

No calls to `new`, `delete`, or heap allocators occur during queue operations. This completely eliminates allocator synchronization overhead, heap fragmentation costs, allocator metadata traffic, and expensive kernel interactions that may occur during memory expansion.

- **No CAS Contention**

In a Single-Producer Single-Consumer architecture, ownership is naturally partitioned:

- Only the producer modifies the tail index
- Only the consumer modifies the head index

Therefore, no expensive Compare-And-Swap retry loops are required. This dramatically reduces cache coherency traffic.

- **Minimal Cache Line Invalidations**

Since producer and consumer operate largely on separate memory regions, the CPU rarely needs to invalidate cache lines across cores. Combined with `alignas(64)` padding, this minimizes false sharing almost entirely.

- **Deterministic Hot Path**

Every operation follows a highly predictable execution pattern:

- No locks
- No allocations
- No syscalls
- Very few branches

This allows branch predictors and out-of-order execution engines to operate extremely efficiently.

BM_SPSC_UNBOUNDED — **6.44 Million Ops/sec**

Despite remaining lock-free, throughput drops drastically compared to the bounded version.

Reason for Slowdown

Although the queue remains lock-free, the transition from an array-backed ring buffer to a linked-list architecture introduces several major performance penalties.

- **Pointer Chasing**

Linked-list nodes are scattered throughout heap memory rather than being contiguous. Traversing these nodes forces the CPU to repeatedly fetch unrelated memory locations, causing frequent cache misses and pipeline stalls.

- **Allocator Overhead**

Every push operation requires allocating a new node dynamically:

```
new node(...)
```

Heap allocators themselves are internally synchronized structures. Under contention, allocator locks and metadata management become substantial bottlenecks.

- **Poor Hardware Prefetching**

CPU prefetchers are highly optimized for predictable sequential memory access patterns. Linked-list traversal destroys this predictability because future nodes cannot be inferred ahead of time.

- **TLB Pressure**

Since allocations may span many memory pages, the Translation Lookaside Buffer (TLB) experiences higher miss rates. Address translation overhead therefore increases.

- **Memory Fragmentation**

Long-running workloads may fragment the heap, reducing locality even further and increasing allocator traversal costs.

The unbounded queue therefore trades raw throughput for flexibility and unlimited capacity growth.

10.2 Multi-Producer Single-Consumer (MPSC)

- BM_MPSC/1 → 5.73M ops/sec
- BM_MPSC/2 → 6.62M ops/sec
- BM_MPSC/4 → 6.93M ops/sec
- BM_MPSC/8 → 6.88M ops/sec

Primary Bottlenecks

As producer count increases, several scalability bottlenecks emerge simultaneously.

- **CAS Retry Storms**

All producers attempt to update the shared tail pointer concurrently using atomic Compare-And-Swap operations.

Under contention:

1. Multiple threads read the same tail value
2. One succeeds in updating it
3. All others fail
4. Failed threads retry repeatedly

This creates heavy retry-loop overhead and wasted CPU work.

- **Cache Coherency Traffic**

The tail pointer becomes a highly contested cache line bouncing continuously between CPU cores. Each successful atomic modification forces other cores to invalidate their local cached copies.

This phenomenon grows increasingly expensive as thread count rises.

- **Memory Allocator Contention**

Since each producer dynamically allocates nodes independently, allocator synchronization overhead scales poorly under concurrent allocation pressure.

- **Single Consumer Limitation**

Regardless of producer count, only one thread ultimately drains the queue. Eventually the consumer becomes the throughput ceiling.

These effects collectively explain why throughput plateaus rather than scaling linearly with thread count.

10.3 Lock-Free MPMC Bounded

- BM_LOCKFREE_MPMC_BOUNDED/2 → 31.87M ops/sec
- BM_LOCKFREE_MPMC_BOUNDED/4 → 9.83M ops/sec
- BM_LOCKFREE_MPMC_BOUNDED/8 → 8.46M ops/sec
- BM_LOCKFREE_MPMC_BOUNDED/16 → 5.75M ops/sec

Analysis

At low thread counts, performance is exceptional because:

- The queue is array-backed
- No dynamic allocation exists
- Memory locality remains strong
- Cache utilization remains efficient

Why Performance Drops

The lock-free MPMC queue scales significantly worse as thread count rises because both enqueue and dequeue operations become globally contested synchronization points.

- **Shared Atomic Hotspots**

Producers compete on the shared tail index while consumers compete on the shared head index. These atomics become serialization bottlenecks under heavy contention.

- **Cross-Core Cache Synchronization**

Modern CPUs maintain cache coherency using protocols such as MESI. Every atomic write forces ownership transfer of cache lines between cores.

As thread count increases:

- Cache invalidations increase dramatically
- Inter-core interconnect traffic rises
- Latency grows non-linearly

- **Memory Ordering Costs**

Acquire-release synchronization semantics introduce additional fencing and ordering constraints that partially restrict CPU instruction reordering optimizations.

- **Retry Amplification**

Failed CAS operations become more frequent at high thread counts, meaning threads spend increasing amounts of time spinning rather than completing useful work.

- **NUMA Effects**

On larger multi-socket systems, cache line ownership transfers may even cross NUMA boundaries, increasing latency substantially further.

This demonstrates an important principle of concurrent systems:

Lock-free does not imply contention-free

10.4 Blocking MPMC

- `BM_BLOCKING_MPMC/2` → 3.16M ops/sec
- `BM_BLOCKING_MPMC/4` → 2.44M ops/sec
- `BM_BLOCKING_MPMC/8` → 1.55M ops/sec
- `BM_BLOCKING_MPMC/16` → 1.10M ops/sec

Main Performance Costs

Blocking queues trade throughput for reduced CPU utilization and improved scheduling fairness. Their primary overheads arise from operating system interaction.

- **Mutex Contention**

Threads attempting to acquire an already-held mutex must block and enter the operating system scheduler queue.

- **Kernel Context Switching**

Suspending and waking threads requires saving and restoring:

- CPU registers
- Instruction pointers
- Stack state
- Scheduler metadata

Context switches can cost thousands of CPU cycles.

- **Scheduler Overhead**

The operating system must continually decide:

- Which thread runs next
- Which thread sleeps
- Which thread wakes

Scheduler overhead becomes increasingly severe at high thread counts.

- **Lock Convoys**

When many threads wait for the same mutex, execution may serialize into a convoy pattern where threads wake sequentially only to block again shortly afterward.

- **Poor Cache Warmth**

Sleeping threads lose cache locality while descheduled. When awakened later, much of their working set may no longer reside in CPU caches.

Despite lower throughput, blocking queues remain extremely practical because they avoid wasting CPU resources while idle.

11 Final Takeaways

- **Memory layout is often more important than algorithmic complexity.**

Contiguous ring buffers massively outperform linked structures due to cache locality, predictable access patterns, and hardware prefetching.

- **Lock-free does not guarantee perfect scalability.**

Even without mutexes, shared atomic variables eventually become serialization bottlenecks due to cache coherency traffic and CAS contention.

- **Dynamic allocation is extremely expensive in concurrent systems.**

Heap allocators introduce synchronization overhead, fragmentation, TLB pressure, and poor cache locality.

- **CPU architecture heavily influences concurrent performance.**
Factors such as cache hierarchy, MESI coherency protocols, memory fences, NUMA topology, and branch prediction all significantly impact throughput.
- **Different queue designs optimize for different goals.**
 - Bounded lock-free queues optimize raw throughput
 - Unbounded queues optimize flexibility
 - Blocking queues optimize CPU efficiency
- **Acquire-release memory ordering provides a major performance advantage.**
Carefully relaxing memory ordering constraints avoids unnecessary full memory fences while still preserving correctness.
- **False sharing can completely destroy scalability.**
Proper cache-line alignment and padding are essential in high-performance concurrent structures.